VERILOG ASSIGNMENT ON RISC

GROUP - 55

UTSA GHOSH (23CS10090) AND

SOUHARDYA DANDAPAT (23CS30054)

Types of instructions and the opcodes and function codes :

1. R type Instructions:

Opcode	rs	rt	rd	shamt	function	Unused
6 bits	4 bits	4 bits	4 bits	5 bits	6 bits	3 bits

Instructions	Opcode	Function
ADD	000000	001000
SUB	000000	001001
INC	000000	001010
DEC	000000	001011
SLT	000000	001100
SGT	000000	001101
AND	000000	010000
OR	000000	010001
XOR	000000	010010
NOR	000000	010011
NOT	000000	010100
SLA	000000	011011
SRL	000000	011001
SRA	000000	011010
HAM	000000	101000
MOVE	000000	110000
CMOV	000000	110001

2. I type Instructions:

opcode	rs	rt	immediate	Unused
6 bits	4 bits	4 bits	16 bits	2 bits

Instruction	Opcode
ADDI	001000
SUBI	001001
SLAI	011000
SRLI	011001
SRAI	011010
ANDI	010000
ORI	010001
XORI	010010
LUI	110000
LD	101010
ST	101011
BMI	111001
BPL	111010
BZ	111011

3. J type Instructions:

opcode	Immediate	Unused
6 bits	26 bits	0 bits

Instruction	Opcode
BR	111000

Program Control Instructions:

Instruction	Opcode
HALT	101000
NOP	101001

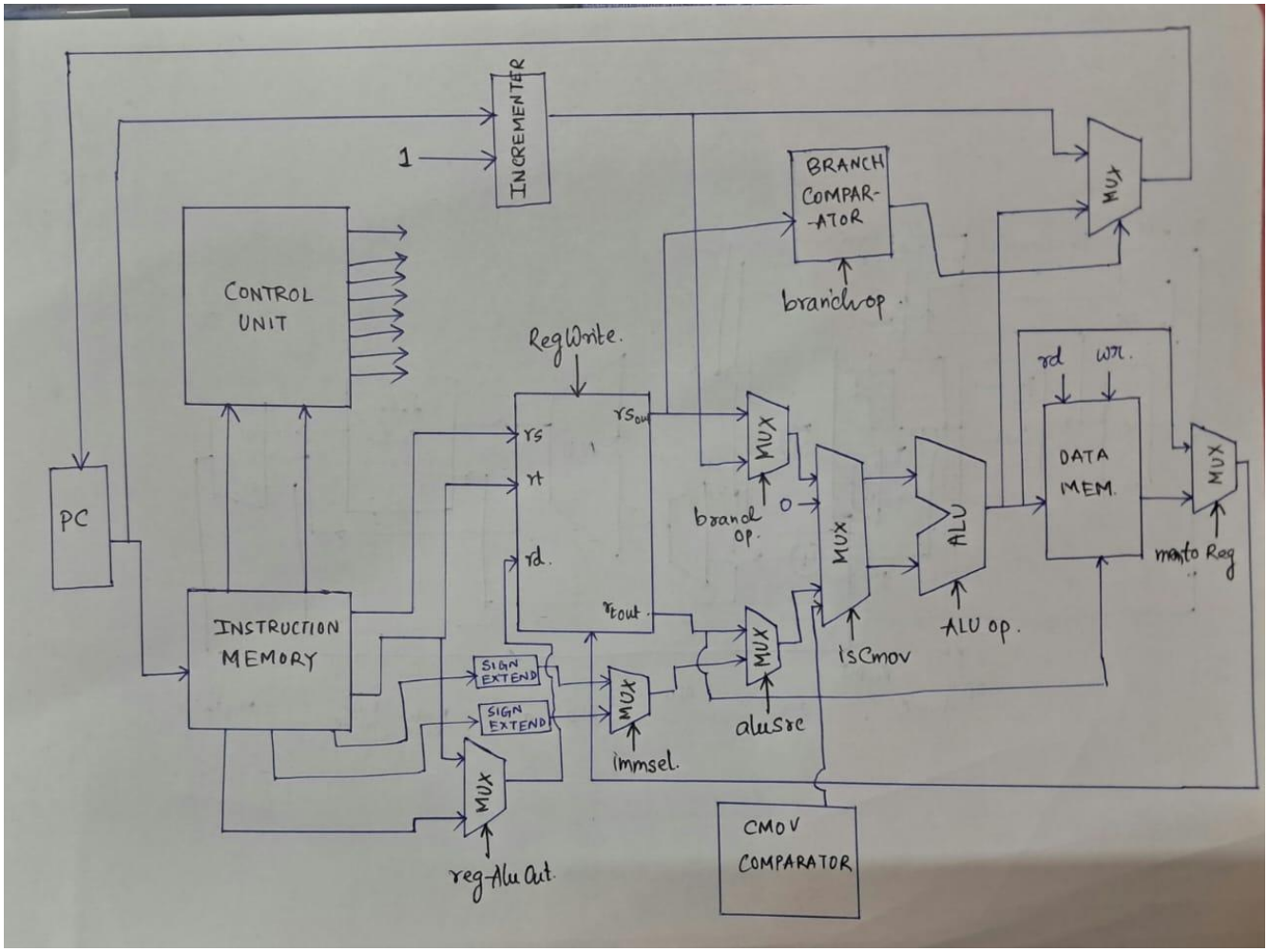

Fig : DataPath

Assumptions & Reasons for the Opcode/Function Scheme :

Assumption 1 – R-type uses opcode = 000000, operation selected by funct.

Reasons : Reserving the all-zero opcode for R-type ensures that ALU operations are uniquely identified by the funct field, avoiding conflicts with memory, branch, and control instruction spaces. This simplifies the opcode map and preserves opcode space for non-ALU instruction classes, while also making decoding straightforward (R-type instructions are immediately recognizable).

Assumption 2 – Top 3 bits encode the instruction class, lowest 3 bits encode the specific operation within the class.

Reasons: Partitioning the 6-bit field into [5:3] = *class* and [2:0] = *member* provides a uniform structure across R/I/other formats.

The **class field** ([5:3]) enables early decoding and routing (e.g., ALU, Shift, Logic, Memory, Branch, Control).

The **member field** ([2:0]) consistently encodes the specific function inside each class (e.g., Logic class: 000 = AND, 001 = OR, 010 = XOR, etc.).
This uniform scheme allows a common decoder structure, reducing hardware complexity and ensuring consistency across instruction types.

Assumption 3 – Class-member codes are consistent between R-type (funct) and I-type (opcode).

Reasons : A given 6-bit pattern represents the same operation regardless of format. For example, 010000 denotes **AND** in R-type (funct) and **ANDI** in I-type (opcode). This symmetry allows reuse of the same decode logic for both immediate and register-based variants, minimizing duplication in the control unit and ensuring uniformity in the instruction set architecture.

Note 1 – Branch Operations
Branch instructions are handled by a dedicated block that uses a *branch operation selection flag*. This flag determines the specific branch condition (e.g., BMI, BPL, BZ, etc.) and directs the hardware to perform the corresponding branch evaluation.

Note 2 – Conditional Move (CMOV)
A comparator block is employed to detect CMOV instructions. If the instruction is identified as CMOV and the comparator output (mux input) evaluates to logic 1 (true), the conditional move operation is executed. Otherwise, the move is not performed, ensuring correctness of conditional execution.

Note 3 – ALU op is broken into two parts :

1. FuncSel and 2. BlockSel

FuncSel selects the function to be performed by the ALU(like various functions in Arithmetic, Shifter, Logical, LUI, HAM) and Block select selects the category in which the operation falls.

Hardwire Control Signals:

Note: Here we have followed the following notation: + = OR, . = AND, ~ = NOT.

Inputs & R-type detector

- **op[5:0] = (op5 op4 op3 op2 op1 op0)**
- **func[5:0] = (f5 f4 f3 f2 f1 f0)**
- **R = ~op5~op4~op3~op2~op1~op0**

Decoders:

MSB equalities (for func):

F_BS001 = ~f5~f4f3

F_BS010 = ~f5f4~f3

F_BS011 = ~f5f4f3

F_BS100 = f5~f4~f3

F_BS101 = f5~f4f3

F_BS110 = f5f4~f3

F_BS111 = f5f4f3

MSB (for op):

O_BS001 = ~op5~op4op3

O_BS010 = ~op5op4~op3

O_BS011 = ~op5op4op3

O_BS100 = op5~op4~op3

O_BS101 = op5~op4op3

O_BS110 = op5op4~op3

O_BS111 = op5op4op3

LSB (for func):

F_FS000 = ~f2~f1~f0

F_FS001 = ~f2~f1f0

F_FS010 = ~f2f1~f0

F_FS011 = ~f2f1f0

LSB (for op):

O_FS000 = ~op2~op1~op0

O_FS001 = ~op2~op1op0

O_FS010 = ~op2op1~op0

O_FS011 = ~op2op1op0

ALU function select (to ALU)

aluFn2 = (R f2) + (~R op2)

aluFn1 = (R f1) + (~R op1)

aluFn0 = (R f0) + (~R op0)

ALU Block select (to ALU)

aluBlk2 = (R f5) + (~R op5)

aluBlk1 = (R f4) + (~R op4)

aluBlk0 = (R f3) + (~R op3)

Register write enable

RegWrite = R + ~R[O_BS001 + O_BS010 + O_BS011 + O_BS110 + O_BS101 . O_BS010]

ALU B-input source (1 = immediate, 0 = register)

aluSrc = ~R + R F_BS110 (F_FS000 + F_FS001)

Equivalent compact form: aluSrc = ~R + R (f5~f4f3~f2~f1)

Immediate format select (branch's special immediate)

immSel = (~R O_BS111 O_FS000)

Memory controls and writeback

rd = (~R O_BS101 O_FS010) (Load)

wr = (~R O_BS101 O_FS011) (Store)

memToReg = rd

Conditional move flag

isCmov = R F_BS110 F_FS001

Branch/jump operation code (brOp[2:0])

brOp2 = (R) + (~R ~(O_BS111 O_FS001 + O_BS111 O_FS010 + O_BS111 O_FS011 + O_BS111 O_FS000))

brOp1 = (~R (O_BS111 O_FS010 + O_BS111 O_FS011))

brOp0 = (~R(O_BS111 O_FS001 + O_BS111 O_FS011))

Register-ALU output latch enable

regAluOut = R